

WRITING QUALITY SOFTWARE - SOLID PRINCIPLES AND CODE SMELLS

SE3070 - CASE STUDIES IN SOFTWARE ENGINEERING

SEMESTER 2, 2025

SLIIT

A decorative horizontal bar at the bottom of the slide with a gradient from red on the left to purple on the right.

LEARNING OUTCOMES

- After completing this topic, you will be able to,
 - Explain why good designs can fail during implementation.
 - Apply SOLID principles to improve code quality.
 - Define what a code smell is.
 - Identify and explain some key code smells in OO systems.
 - Relate code smells to underlying design or implementation flaws.

CONTENTS

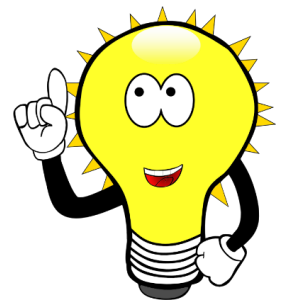
- Design vs. Implementation
- Design vs. Implementation Considerations
- Why can Implementations fail?
- Historical Approaches to Reduce Failure
- SOLID Principles for Good Implementation
- Code Smells
 - What is a Code Smell?
 - Common Code Smells
- Summary

DESIGN VS. IMPLEMENTATION CONSIDERATIONS

- Design
 - UX, Architecture, responsibilities, and high-level interactions.
- Implementation
 - Code structure, library use, frameworks
- Even perfect designs can fail with poor implementation.
 - Example: Well-structured classes implemented with duplication.

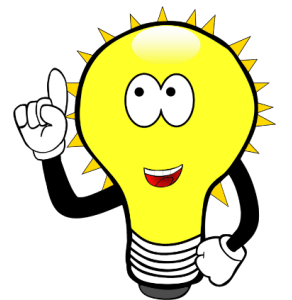
DESIGN VS. IMPLEMENTATION

- Software design defines the structure and interactions of components.
- Good design is essential, but implementation choices can undermine it.
- Implementation turns the design into working code.
- Our goal is to bridge the gap between design and implementation quality.



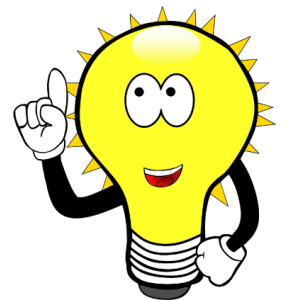
WHY IMPLEMENTATION CAN FAIL

- Misinterpretation of design by developers.
- **Time pressure and shortcuts.**
 - [\[Youtube\] Knight Capital loses \\$440 million in 30 mins](#)
- Lack of code review or quality control.
- Poor understanding of design principles.
- Overcomplication during coding phase.



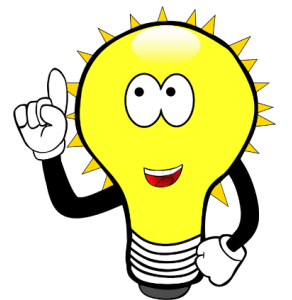
HISTORICAL APPROACHES TO REDUCE FAILURE

- Structured Programming
 - Clear control flow.
- Procedural Design Principles
 - Modular procedures.
- Early OO Guidelines
 - Encapsulation, inheritance, and polymorphism.
- SOLID principles
 - [SOLID Principles – Uncle Bob](#)



SOLID PRINCIPLES FOR GOOD IMPLEMENTATION

- **S**ingle Responsibility Principle (SRP)
- **O**pen/Closed Principle (OCP)
- **L**iskov Substitution Principle (LSP)
- **I**nterface Segregation Principle (ISP)
- **D**ependency Inversion Principle (DIP)



SINGLE RESPONSIBILITY PRINCIPLE



SINGLE RESPONSIBILITY PRINCIPLE

Just Because You Can, Doesn't Mean You Should

Source: <https://www.tomdalling.com/blog/software-design/solid-class-design-the-single-responsibility-principle/>

SINGLE RESPONSIBILITY PRINCIPLE

- A class should have one, and only one, reason to change (i.e. high cohesion)
- Why bad to violate this principle?
 - Too many responsibilities.
 - Codebase is harder to read and understand
 - Fragile codebase - harder to maintain and test.
 - Bad example on the right.

```
class Order {  
    // order details...  
}  
  
class OrderService {  
    void createOrder(Order o) {  
        System.out.println("Order created");  
    }  
  
    void printInvoice(Order o) {  
        System.out.println("Invoice printed");  
    }  
  
    void sendEmailConfirmation(Order o) {  
        System.out.println("Email confirmation sent");  
    }  
}
```

SINGLE RESPONSIBILITY PRINCIPLE

```
class Order {  
    // order details...  
}  
  
class OrderService {  
    void createOrder(Order o) {  
        System.out.println("Order created");  
    }  
}
```

```
class InvoicePrinter {  
    void printInvoice(Order o) {  
        System.out.println("Invoice printed");  
    }  
}  
  
class EmailService {  
    void sendEmailConfirmation(Order o) {  
        System.out.println("Email confirmation sent");  
    }  
}
```

Better way of handling things preserving SRP.

SINGLE RESPONSIBILITY PRINCIPLE

- Benefits of SRP:
 - Code is easier to be understood, debugged and refactored.
 - The class is easier to maintain.
 - Changes are isolated - if you break something, you break one thing (or fewer), not an entire system.
 - The class is more reusable.
 - Read more [here](#).

OPEN/CLOSED PRINCIPLE



Source: <https://www.tomdalling.com/blog/software-design/solid-class-design-the-open-closed-principle/>

OPEN/CLOSED PRINCIPLE

- A class should be open for extension, closed for modification.
- Why bad to violate this principle?
 - Existing code needs to be modified to do any change resulting in code stability issues.
 - Fragile codebase - harder to maintain and test.
 - Bad example on the right.

```
class Order {
    String customerType;

    public Order(String customerType) {
        this.customerType = customerType;
    }
}

class DiscountService {
    double getDiscount(Order o) {
        if (o.customerType.equals("REGULAR")) {
            return 0.05;
        } else if (o.customerType.equals("PREMIUM")) {
            return 0.10;
        }
        // ❌ Adding a new customer type requires modifying this method
        return 0.0;
    }
}
```

OPEN/CLOSED PRINCIPLE

```
interface DiscountPolicy {  
    double getDiscount();  
}  
  
class RegularCustomerDiscount implements DiscountPolicy {  
    @Override  
    public double getDiscount() {  
        return 0.05;  
    }  
}  
  
class PremiumCustomerDiscount implements DiscountPolicy {  
    @Override  
    public double getDiscount() {  
        return 0.10;  
    }  
}
```

```
class Order {  
    private final DiscountPolicy discountPolicy;  
  
    public Order(DiscountPolicy discountPolicy) {  
        this.discountPolicy = discountPolicy;  
    }  
  
    public double getDiscount() {  
        return discountPolicy.getDiscount();  
    }  
}
```

```
class VipCustomerDiscount implements DiscountPolicy {  
    @Override  
    public double getDiscount() {  
        return 0.15;  
    }  
}
```

Adding a new discount policy called *VIPCustomerDiscount* is a breeze. No changes to existing code - we extend, not modify.

Better way of handling things preserving OCP.

OPEN/CLOSED PRINCIPLE

- The Open-Closed Principle requires that classes should be open for extension and closed to modification.
- Modification means changing the code of an existing class, and extension means adding new functionality.
 - We should be able to add new functionality without touching the existing code for the class.
 - It is usually done with the help of interfaces and abstract classes.
- Benefits?
 - As no modifications are made to the code no bugs can be introduced, leading to code that only **becomes more stable over time** through testing.
 - The ability to reuse a class that has been working for years without modification is clearly preferable to modifying the class every time requirements change.

LSKOV SUBSTITUTION PRINCIPLE



LSKOV SUBSTITUTION PRINCIPLE

No matter what you used to learn how to drive, you should be able to drive any car afterward.

Source: <https://blog.larapulse.com/clean-code/solid-in-simple-words/>

LSKOV SUBSTITUTION PRINCIPLE

- Subtypes must be substitutable for their base types.
- Why bad to violate this principle?
 - Breaks expectations on child behavior.
 - Whole point in using an abstract base class is so that, in the future, you can write a new subclass and insert it into existing, working, tested code.
 - Bad example on the right.

```
class Order {  
    void applyDiscount(double percentage) {  
        System.out.println("Discount applied: " + percentage + "%");  
    }  
}  
  
// ✓ LSP-preserving: still applies a discount as expected  
class SeasonalOrder extends Order {  
    @Override  
    void applyDiscount(double percentage) {  
        double totalDiscount = percentage + 5; // seasonal bonus  
        System.out.println("Discount applied: " + totalDiscount + "% (including seasonal bonus)");  
    }  
}  
  
// ✓ LSP-preserving: still applies a discount as expected  
class BulkOrder extends Order {  
    @Override  
    void applyDiscount(double percentage) {  
        double totalDiscount = percentage + 10; // bulk bonus  
        System.out.println("Discount applied: " + totalDiscount + "% (bulk discount included)");  
    }  
}  
  
// ✗ LSP-violating: breaks expected behavior by throwing  
class FreeOrder extends Order {  
    @Override  
    void applyDiscount(double percentage) {  
        throw new UnsupportedOperationException("Free orders can't have discounts");  
    }  
}
```

LSKOV SUBSTITUTION PRINCIPLE

- Subtypes must be substitutable for their base types.
- Why bad to violate this principle?
 - Breaks expectations on child behavior.
 - Whole point in using an abstract base class is so that, in the future, you can write a new subclass and insert it into existing, working, tested code.
 - Bad example on the right.

```
class Order {  
    void applyDiscount(double percentage) {  
        System.out.println("Discount applied: " + percentage + "%");  
    }  
}  
  
class FreeOrder extends Order {  
    @Override  
    void applyDiscount(double percentage) {  
        // Free orders should not have discounts,  
        // but throwing here breaks LSP.  
        throw new UnsupportedOperationException("Free orders can't have discounts");  
    }  
}  
  
Order order = getOrderFromSomewhere();  
order.applyDiscount(10);  
  
Order order = new FreeOrder();  
order.applyDiscount(10); // 💣 Runtime error!
```

LSKOV SUBSTITUTION PRINCIPLE

```
interface Discountable {  
    void applyDiscount(double percentage);  
}  
  
class Order implements Discountable {  
    @Override  
    public void applyDiscount(double percentage) {  
        System.out.println("Discount applied: " + percentage + "%");  
    }  
}  
  
class FreeOrder {  
    // No discount-related behavior  
    void markAsFree() {  
        System.out.println("Order is free");  
    }  
}
```

Better way of handling things preserving LSP.

FreeOrder is not pretending to be an Order that supports discounts, so there's no LSP violation. All other LSP preserving classes can implement this interface. Any code that deals with *Discountable* can safely assume the method behaves the same always.

INTERFACE SEGREGATION PRINCIPLE



Source: <https://blog.larapulse.com/clean-code/solid-in-simple-words/>

INTERFACE SEGREGATION PRINCIPLE

- **Clients should not be forced to depend on methods they do not use.**
- Many client-specific interfaces are better than one general-purpose interface.
- Why bad to violate this principle?
 - Some classes may not need to implement all the stubs.
 - “*Fat Interfaces*” - interfaces with additional useless methods lead to more coupling.
 - Bad example on the right.

```
interface OrderOperations {  
    void createOrder();  
    void cancelOrder();  
    void shipOrder();  
    void trackOrder();  
}  
  
class DigitalOrder implements OrderOperations {  
  
    @Override  
    public void createOrder() {  
        System.out.println("Digital order created");  
    }  
  
    @Override  
    public void cancelOrder() {  
        System.out.println("Digital order cancelled");  
    }  
  
    @Override  
    public void shipOrder() {  
        // ✗ Digital products aren't shipped!  
        throw new UnsupportedOperationException("Digital orders cannot be shipped");  
    }  
  
    @Override  
    public void trackOrder() {  
        // ✗ Tracking makes no sense for digital downloads  
        throw new UnsupportedOperationException("No tracking available for digital orders");  
    }  
}
```

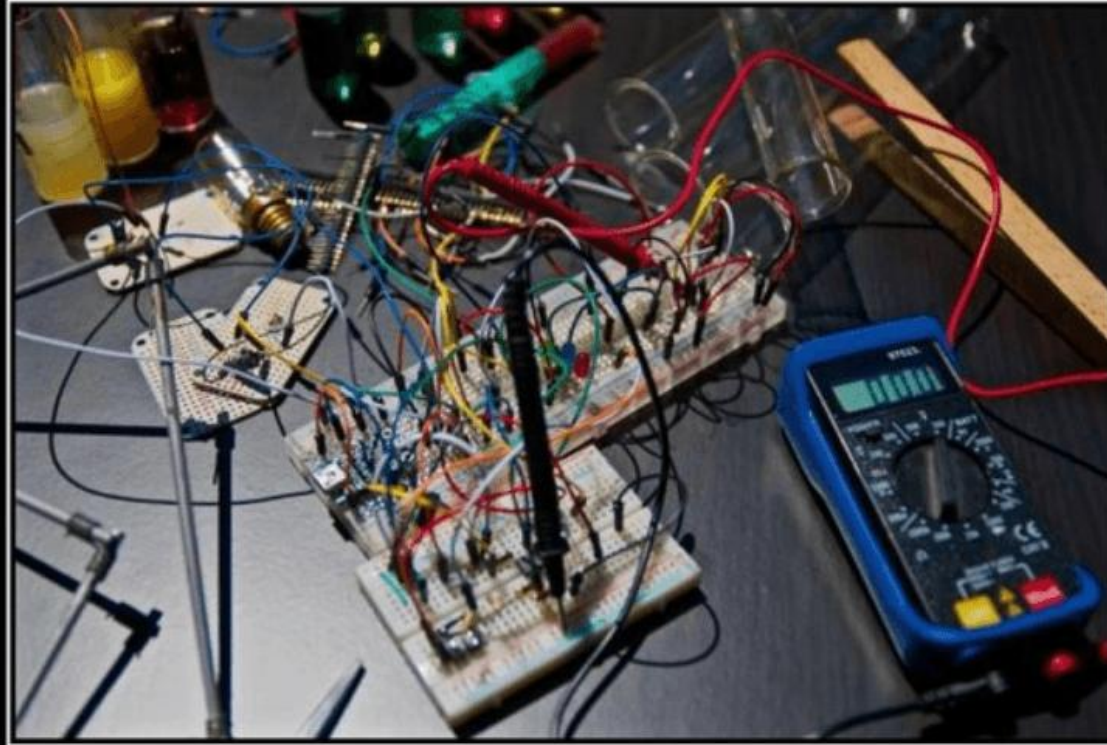
INTERFACE SEGREGATION PRINCIPLE

```
interface OrderCreation {  
    void createOrder();  
    void cancelOrder();  
}  
  
interface Shippable {  
    void shipOrder();  
    void trackOrder();  
}  
  
class DigitalOrder implements OrderCreation {  
  
    @Override  
    public void createOrder() {  
        System.out.println("Digital order created");  
    }  
  
    @Override  
    public void cancelOrder() {  
        System.out.println("Digital order cancelled");  
    }  
}
```

```
class PhysicalOrder implements OrderCreation, Shippable {  
  
    @Override  
    public void createOrder() {  
        System.out.println("Physical order created");  
    }  
  
    @Override  
    public void cancelOrder() {  
        System.out.println("Physical order cancelled");  
    }  
  
    @Override  
    public void shipOrder() {  
        System.out.println("Physical order shipped");  
    }  
  
    @Override  
    public void trackOrder() {  
        System.out.println("Tracking physical order");  
    }  
}
```

Better way of handling things preserving ISP.

DEPENDENCY INVERSION PRINCIPLE



DEPENDENCY INVERSION PRINCIPLE

You don't have to learn how to wire things up if you need to turn on the lights. You use a socket for that.

Source: <https://blog.larapulse.com/clean-code/solid-in-simple-words/>

DEPENDENCY INVERSION PRINCIPLE

- High level modules should not depend upon low level modules. Both should depend upon abstractions.
- Why bad to violate this principle?
 - Tightly coupled to one concrete class.
 - Swapping to another concrete class requires code modification (OCP violation).
 - Testing becomes harder (harder to mock a module)

```
class EmailService {  
    public void sendEmail(String message) {  
        System.out.println("Sending email: " + message);  
    }  
}  
  
class OrderProcessor {  
    // ❌ High-Level module depends directly on a low-level module  
    private EmailService emailService = new EmailService();  
  
    public void processOrder() {  
        System.out.println("Processing order...");  
        emailService.sendEmail("Order processed");  
    }  
}
```

DEPENDENCY INVERSION PRINCIPLE

```
interface NotificationService {  
    void sendNotification(String message);  
}  
  
class EmailService implements NotificationService {  
    @Override  
    public void sendNotification(String message) {  
        System.out.println("Sending email: " + message);  
    }  
}  
  
class SMSService implements NotificationService {  
    @Override  
    public void sendNotification(String message) {  
        System.out.println("Sending SMS: " + message);  
    }  
}
```

```
class OrderProcessor {  
    private final NotificationService notificationService;  
  
    //  Dependency injected via constructor  
    public OrderProcessor(NotificationService notificationService) {  
        this.notificationService = notificationService;  
    }  
  
    public void processOrder() {  
        System.out.println("Processing order...");  
        notificationService.sendNotification("Order processed");  
    }  
}
```

Better way of handling things preserving DIP.

CODE SMELLS

```
public abstract class AbstractCollection implements Collection {  
    public void addAll(AbstractCollection c) {  
        if (c instanceof Set) {  
            Set s = (Set)c;  
            for (int i=0; i < s.size(); i++) {  
                if (!contains(s.getElementAt(i))) {  
                    add(s.getElementAt(i));  
                }  
            }  
        }  
        else if (c instanceof List) {  
            List l = (List)c;  
            for (int i=0; i < l.size(); i++) {  
                if (!contains(l.get(i))) {  
                    add(l.get(i));  
                }  
            }  
        }  
        else if (c instanceof Map) {  
            Map m = (Map)c;  
            for (int i=0; i<m.size(); i++)  
                add(m.keys[i], m.values[i]);  
        }  
    }  
}
```

The diagram illustrates several code smells in the provided Java code snippet. A large black bracket on the right side of the code block groups the entire `addAll` method, with a line pointing to the label "Long Method". Inside this bracket, several specific smells are highlighted with colored callouts and lines pointing to the corresponding code:

- Duplicated Code** (blue text): Points to the `if (!contains(s.getElementAt(i)))` condition in the `Set` branch.
- Duplicated Code** (purple text): Points to the `if (!contains(l.get(i)))` condition in the `List` branch.
- Alternative Classes with Different Interfaces** (green text): Points to the `if (!contains(s.getElementAt(i)))` and `if (!contains(l.get(i)))` conditions, highlighting the inconsistency in how the `contains` method is accessed across different collection types.
- Switch Statement** (red text): Points to the `if (c instanceof Set)`, `else if (c instanceof List)`, and `else if (c instanceof Map)` structure, suggesting a `switch` statement might be more appropriate.
- Inappropriate Intimacy** (red text): Points to the `m.keys[i]` and `m.values[i]` access in the `Map` branch, indicating a lack of abstraction.

Code Smells. Source: <https://itnext.io/design-discussion-code-smell-bb55b19eb1a8>

CODE SMELLS

- What is a Code Smell?
 - Symptom in code that may indicate a deeper problem.
 - Even with SOLID, issues can arise in implementation.
 - Code smells are symptoms, not necessarily bugs.
 - Doesn't necessarily mean that the code is wrong or broken
 - [Read: Code Smell - Martin Fowler](#)
 - Detecting them early prevents technical debt.

CODE SMELLS

- Common Code Smells
 - Long Method
 - Large Class (God Class)
 - Primitive Obsession & Data Clumps
 - Long Parameter List
 - Switch/If-Else Chains
 - Shotgun Surgery
 - Divergent Change
 - Duplicate Code
 - Dead Code
 - Temporary Field
 - Middle Man
 - Feature Envy
 - Message Chains (Law of Demeter Violation)

Read more [here](#).

LONG METHOD

- Definition: A method that is too long and tries to do too much.
- Why Bad?
 - Hard to read and maintain.
 - Any change risks breaking multiple unrelated steps.
 - Bad example on the right.

```
void processOrder(Order o) {  
    validateOrder(o);  
    calculateDiscount(o);  
    generateInvoice(o);  
    sendEmail(o);  
    updateInventory(o);  
    recordTransaction(o);  
    notifyWarehouse(o);  
}
```

LONG METHOD

- Smaller, focused methods improve readability and maintainability.
- Better way of handling this code on the right.

```
void processOrder(Order o) {  
    validateOrder(o);  
    applyDiscount(o);  
    finalizeOrder(o);  
}
```

```
void finalizeOrder(Order o) {  
    generateInvoice(o);  
    sendEmail(o);  
    updateInventory(o);  
    notifyWarehouse(o);  
}
```

LARGE CLASS (GOD CLASS)

- Definition: A class that has too many responsibilities.
- Why Bad?
 - Violates SRP.
 - Hard to maintain, test, or reuse.

```
class OrderManager {  
    void createOrder(Order o) {  
        /* ... */  
    }  
  
    void cancelOrder(Order o) {  
        /* ... */  
    }  
  
    void printInvoice(Order o) {  
        /* ... */  
    }  
  
    void sendEmail(Order o) {  
        /* ... */  
    }  
  
    void updateInventory(Order o) {  
        /* ... */  
    }  
  
    void processRefund(Order o) {  
        /* ... */  
    }  
  
    void generateSalesReport() {  
        /* ... */  
    }  
}
```

LARGE CLASS (GOD CLASS)

- Good Example:

```
class OrderService { void createOrder(Order o) { /* ... */ } }
```

```
class CancellationService { void cancelOrder(Order o) { /* ... */ } }
```

```
class InvoicePrinter { void printInvoice(Order o) { /* ... */ } }
```

```
class EmailService { void sendEmail(Order o) { /* ... */ } }
```

```
class InventoryService { void updateInventory(Order o) { /* ... */ } }
```

```
class RefundService { void processRefund(Order o) { /* ... */ } }
```

```
class SalesReportGenerator { void generateReport() { /* ... */ } }
```

- Responsibilities split into cohesive, independent classes.

PRIMITIVE OBSESSION & DATA CLUMPS

- Definition:
 - Primitive Obsession: Overusing primitive types instead of domain-specific objects.
 - Data Clumps: Groups of variables that are often passed around together.
- Bad Example:

```
class Customer {  
    String firstName;  
    String lastName;  
    String street;  
    String city;  
    String postalCode;  
}
```
- Why Bad:
 - Fields/ properties appear together everywhere, logic duplicated.

PRIMITIVE OBSESSION & DATA CLUMPS

- Good Example:

```
class Address { String street; String city; String postalCode; }  
class Customer { String firstName; String lastName; Address  
address; }
```

- Encapsulation reduces duplication and improves clarity.

LONG PARAMETER LIST

- Definition: Too many parameters in a method.

- Bad Example:

```
void createOrder(String customerName, String street, String city,  
String postalCode, List<Item> items, double discount) { /* ... */ }
```

- Why Bad:

- Hard to read, easy to misorder arguments, difficult to extend.

- Good Example:

```
void createOrder(Customer customer, List<Item> items,  
DiscountPolicy discount) { /* ... */ }
```

- Pass objects instead of many related primitives.

SWITCH/IF-ELSE CHAINS

- Definition: Using **long** conditional chains to determine behavior.
- Bad Example:

```
double getDiscount(Order o) {  
    if (o.customerType.equals("REGULAR")) return 0.05;  
    else if (o.customerType.equals("PREMIUM")) return 0.10;  
    else if (o.customerType.equals("STUDENT")) return 0.15;  
}
```

SWITCH/IF-ELSE CHAINS

- Why Bad?
 - Violates Open/ Closed Principle.
 - Adding new customer types requires editing the same code.
- Good Example:

```
interface DiscountPolicy { double getDiscount(Order o); }  
class RegularDiscount implements DiscountPolicy { double  
getDiscount(Order o) { return 0.05; } }  
class PremiumDiscount implements DiscountPolicy { double  
getDiscount(Order o) { return 0.10; } }  
class StudentDiscount implements DiscountPolicy { double  
getDiscount(Order o) { return 0.15; } }
```
- Easy to extend without modifying existing classes.

SHOTGUN SURGERY

- Definition: One change requires many edits in multiple classes.
- Bad Example:
 - Adding a new tax rule requires updating OrderService, InvoicePrinter, PaymentService, ReportGenerator
- Why Bad:
 - Risk of missing updates.
 - Increases development time and bugs.
- Good Example:
 - `class TaxCalculator { double calculateTax(Order o) { /* ... */ }}`
- Centralizes logic so only one change is needed.

DIVERGENT CHANGE

- Definition: A class changes for multiple, unrelated reasons.
- Bad Example:

```
class OrderProcessor {  
    void processPayment(Order o) { /* ... */ }  
    void generateInvoice(Order o) { /* ... */ }  
    void sendPromotionalEmail(Customer c) { /* ... */ }  
}
```

DIVERGENT CHANGE

- Why Bad:
 - Payment, invoice, and marketing logic are unrelated.
- Good Example:

```
class PaymentProcessor { void processPayment(Order o) { /* ... */  
    }  
}  
  
class InvoiceService { void generateInvoice(Order o) { /* ... */ }  
  
class MarketingService { void sendPromotionalEmail(Customer c)  
    { /* ... */ } }
```
- Classes have one reason to change.

DUPLICATE CODE

- Definition: Same logic appears in multiple places.

- Bad Example:

```
// in OrderService
```

```
double calculateOrderTotal(Order o) { /* ... */ }
```

```
// in ReportGenerator
```

```
double calculateOrderTotal(Order o) { /* ... */ }
```

- Why Bad?

- Bug fixes/ changes require changes in multiple places.

- Good Example:

```
class OrderCalculator { double calculateTotal(Order o) { /* ... */ } }
```

- Centralized logic improves maintainability.

DEAD CODE

- Definition: Code that is never used but still exist in the code base.
- Why Bad?
 - Causes confusion, wastes mental effort during maintenance.
 - Remember the Knight Capital bug?
- Remove unused code.
- Keep history in version control if needed.

TEMPORARY FIELD

- Definition: A field only used under certain conditions.
- Bad Example:

```
class Order { double seasonalDiscount; /* only used for holiday sales */ }
```
- Why Bad?
 - Misleads readers into thinking the field is always relevant. Makes code harder to read.
- Good Example:

```
class SeasonalOrder extends Order { double seasonalDiscount; }
```
- Field exists only where needed.

MIDDLE MAN

- Definition: A class that only delegates work to another class.

- Bad Example:

```
class OrderManager {  
    private OrderService orderService;  
    void createOrder(Order o) { orderService.createOrder(o); }  
}
```

- Why Bad?

- Adds unnecessary indirection without extra value.

- Remove the middle man and call class instances directly.

FEATURE ENVY

- Definition: A method that uses more data from another class than from its own.
- Bad Example:

```
class DiscountService {  
    double calculateDiscount(Order o) {  
        return o.getItems().size() * o.getCustomer().getLoyaltyPoints();  
    }  
}
```

- Why Bad?
 - Knows too much about other class internals, resulting in very tight coupling.
- Good Example:

```
class Order { double calculateDiscount() { /* ... */ }}
```
- Move the behavior to where the data lives.

MESSAGE CHAINS (LAW OF DEMETER VIOLATION)

- Definition: Chaining calls to navigate through multiple objects.
- Bad Example:

```
order.getCustomer().getAddress().getCity();
```
- Why Bad?
 - Breaks encapsulation.
 - Changes in data structure ripple through all callers.
- Good Example:

```
order.getCustomerCity();
```
- Let each object expose only what's needed.

SUMMARY

- Good design can fail if poorly implemented.
- SOLID principles guide better coding decisions.
- Code smells are indicators of deeper design or implementation issues.
- Detecting and fixing smells early reduces technical debt.
- Refactoring is a continuous process.

REFERENCES & FURTHER READING

- [Tom Dalling - Category: Software Design](#) (Some articles on SOLID which you can read).
- [Code Smells - refactoring.guru](#) (mandatory reference).
- [Case Study 4: The \\$440 Million Software Error at Knight Capital](#) (Detailed case study on the Knight Capital bug)

ACKNOWLEDGMENTS

- Some images and slides were taken from previous SE3070 – CSSE Lecture slide deck on SOLID Principles by Mr. Udara Samaratunge.

THANK YOU!

Vishan Jayasinghearachchi

*Department of Software
Engineering*

vishan.j@sliit.lk

